OFFICIAL TECHNICAL DOCUMENTATION

SNNU AI Web Portal

Scientific Portal featuring Local AI Translation (Zero-Data-Leakage), WebXR/Unity Virtual Tours & Automated Bibliometric Pipelines

Institutions: Saarland University (Faculty of Medicine, Homburg) & HTW Saar (Faculty of Engineering, Saarbrücken)

Research Unit: Systems Neuroscience & Neurotechnology Unit (SNNU)

Tech Stack: React 19 + Vite 8 SPA | FastAPI (Python 3.10+) | SQLite | Cloudflare Workers | Local Ollama LLM Engine

Path on Disk: `/home/hairex/Documentos/Drive/Dokumente/Antigravity/SNNU`

Date: October 2026

1. Executive Summary & Project Mission

SNNU (*Systems Neuroscience & Neurotechnology Unit*) is the official interactive web portal of the interdisciplinary research unit co-founded between the **Faculty of Medicine at Saarland University (UdS)** in Homburg/Saar and the **Faculty of Engineering at the Hochschule für Technik und Wirtschaft des Saarlandes (HTW Saar)** in Saarbrücken, Germany.

Led by **Prof. Dr. Daniel J. Strauss** and **Dr. Farah I. Corona-Strauss**, the unit conducts cutting-edge research across multiscale systems neuroscience, computational neuromodeling, neuroergonomics, functional neuroimaging, and sensory neurotechnology (such as cochlear implants and immersive acoustic spaces).

Core Technological Innovations of the Portal

1. **Dynamic Local AI Translation with Zero-Data-Leakage:** Complies with stringent European health data and privacy standards (GDPR). Rather than delegating translations to commercial cloud APIs (e.g., Google Translate), SNNU connects to local or private LAN instances of **Ollama** running open-source models (such as `translategemma:4b` or `qwen2.5:1.5b`). The backend preserves structural HTML tags, hyperlinks, dates, author names, and scientific citations intact.
2. **Immersive WebXR & Unity WebGL Virtual Environments:** Delivers an interactive 360° virtual tour of the research facilities and the 46-speaker acoustic simulation lab, backed by a Cloudflare Worker edge proxy for high-volume binary streaming.
3. **Automated Bibliometric & Publication Pipelines:** Integrates ETL tools that audit academic records against **Crossref** and **OpenAlex** APIs, resolving DOIs and downloading Open Access PDF copies mapped directly to author profiles.

2. General System Architecture

The SNNU portal decouples functionality across four distinct layers to ensure resilience and high response rates:

Layer	Primary Technologies	Responsibilities
Frontend SPA	React 19, Vite 8, Modern CSS3	Single Page Application (SPA). 15-language dynamic selector, floating navigation Dock, detailed views for 6 research labs, researcher profile modals with canonical publication filtering, and Unity WebGL viewport.

Layer	Primary Technologies	Responsibilities
Backend REST API	FastAPI, Uvicorn, SQLAlchemy	High-concurrency async service. Handles HTML parsing and text tokenization, batch translation with Ollama, in-memory LRU language caching, and database configuration via SQLite (`snnu.db`).
Local AI Engine	Ollama Server (Localhost / LAN)	Deterministic translation inference with strict JSON schemas and asyncio semaphore-based concurrency control.
Edge Infrastructure	Cloudflare Workers (`worker.js`)	Serves bundled static assets (`frontend/dist`) and functions as a CORS-enabled streaming proxy with HTTP Range support for Unity binaries (`.data`, `.wasm`) and 360° MP4 video files.

3. Repository Directory Structure

```
SNNU/

├── backend/                        # REST API & translation engine

│   ├── base_data/                  # Seed dictionary structures (Python)

│   │   ├── __init__.py             # Master INITIAL_PAGE_DATA aggregator

│   │   ├── hero.py                 # Hero section and header video config

│   │   ├── home.py                 # Mission, 3 research pillars, methods, timeline

│   │   ├── nav.py                  # Navigation labels and floating dock items

│   │   ├── news.py                 # Scientific milestones and lab news

│   │   ├── people.py               # Personnel directory, photos, biographies

│   │   ├── research_site.py        # Detailed content for the 6 research labs

│   │   └── sections/               # Publications, projects, teaching, preprints, vacancies

│   ├── main.py                     # FastAPI server and translation pipeline

│   ├── models.py                   # ORM models (School, Language, Audience)

│   ├── schemas.py                  # Pydantic validation schemas

│   ├── database.py                 # SQLAlchemy SQLite connection setup
```

```
│   ├── seed.py                    # Database table seeder
│   └── snnu.db                    # SQLite database file
├── frontend/                      # Client SPA application (React 19 + Vite 8)
│   ├── public/                    # Public static assets
│   │   ├── Build/                 # Unity WebGL binaries (UnityLoader.js, data, wasm)
│   │   ├── image/                 # Staff and lab facility photography
│   │   ├── publication/           # Open Access academic publication PDFs
│   │   └── webxr/                 # Interactive environment for VR headsets
│   ├── src/                       # React source code
│   │   ├── components/            # HomePage, ProfileModal, ResearchLabDetail
│   │   ├── App.jsx                # Root component, state orchestrator, i18n
│   │   ├── App.css                # Global styling and floating dock
│   │   └── main.jsx               # React DOM mounting entry point
│   ├── package.json               # Dependencies (React 19.2.7, Vite 8.1.1)
│   └── vite.config.js             # Vite bundler configuration
├── tools/                         # Automation and ETL tools
│   ├── audit_publications.py      # Crossref + OpenAlex scraper -> manifest.csv
│   └── sync_people_photos.py      # Staff portrait scraper from university website
├── manifest.csv                   # Master publication index with local PDF audit status
├── papers.csv                     # Raw DOI list for automated audits
├── worker.js                      # Cloudflare Worker for routing and asset proxying
├── wrangler.jsonc                 # Cloudflare Workers deployment configuration
└── start_demo.sh                  # Simultaneous dual-process startup script
```

4. Frontend: Components & Internationalization (React 19)

4.1 Global State & Orchestration (`App.jsx`)

- **Data State (`page`):** Full hierarchical tree supplied by the backend (`/default-data` or `/generate`). If the backend is offline, the client degrades gracefully to the bundled English fallback.
- **Dynamic Language Selector (15 Languages):** Supports English (`en`), German (`de`), Spanish (`es`), French (`fr`), Italian (`it`), Portuguese (`pt`), Chinese (`zh`), Japanese (`ja`), Korean (`ko`), Russian (`ru`), Arabic (`ar`), Turkish (`tr`), Polish (`pl`), Dutch (`nl`), and Swedish (`sv`).
- **Ghost Title:** Sticky top bar that reveals itself smoothly when the active section title scrolls past the viewport boundary.
- **Author Disambiguation (`canonicalAuthorName`):** Normalized name-matching algorithm mapping variants such as `"d strauss"`, `"d j strauss"`, and `"daniel j strauss"` to canonical `"Daniel J. Strauss"` to accurately attribute publications in modal profiles.

4.2 Key View Components

- **`HomePage.jsx`:** Highlights the unit's three core research pillars:
  1. *Multiscale Systems Neuroscience:* Cortical dynamics from single neurons to large-scale networks.
  2. *Quantitative Neuromodeling:* Biophysical modeling and computational simulations of sensory processing.
  3. *Neuroimaging:* High-density EEG and functional imaging techniques.

  Includes an interactive history timeline (2005: Homburg University Hospital, 2012: Saarbrücken Technikum, 2013: Haptics Lab).
- **`ResearchLabDetail.jsx`:** Dedicated views for the 6 research labs:
  - **Translational Lab:** Directed by Dr. Corinna Martin (clinical neurotechnologies).
  - **Mindscan Lab:** Directed by Dr. Farah I. Corona-Strauss (non-invasive brain monitoring).
  - **Multiscale Lab:** Directed by Dr. Lars Haab (neuromodeling and signal analysis).
  - **Digital Realities Lab:** Directed by Tobias Grün, M.Sc. (VR and human interaction).
  - **Neurocognitive Haptics Lab:** Tactile perception and haptic interfaces.
  - **Neurocognitive Driving Test Field:** Driving simulator neurocognitive evaluation.
- **`ProfileModal.jsx`:** Comprehensive researcher profile modal with portrait, university affiliation, biography, and searchable publication list with direct PDF downloads.
- **Unity WebGL Viewport (360° Tour):** Renders interactive 360° virtual tours fed through Cloudflare Worker streaming endpoints, with direct links to WebXR VR headset modes.

5. Backend: Deterministic Translation Engine (FastAPI + Ollama)

5.1 Translation Pipeline Architecture (`backend/main.py`)

The core engineering achievement of the backend is translating deeply nested JSON payloads without breaking HTML markup, citations, or metadata:

``` Original JSON Tree │ ▼ collect_translatable_values() ──► Filters out keys (id, urls, emails, photos, authors) │ and proper researcher names ▼ split_html_text() ──► Separates prose text from HTML tags () │ Constructs an indexed positional template ▼ request_translations() ──► Sends batches to Ollama using Structured Outputs Schema: │ {"translations": ["text1", "text2", ...]} ▼ [Validation & Bisection] ──► If Ollama response fails or item count mismatches: │ Recursive bisection (splits batch in half and retries) ▼ rebuild_html_text() ──► Re-injects translated prose into original HTML tags │ without modifying CSS classes, tags, or hrefs ▼ In-Memory LRU Cache + JSON Output ```

> **Key Architectural Features**

- **Metadata Isolation (`NON_TRANSLATABLE_KEYS`):** Keys such as `id`, `icon`, `videoSrc`, `src`, `url`, `href`, `email`, `photo`, `phone`, `year`, and `authors` are never dispatched to the LLM.
- **HTML Integrity Preservation:** If a string contains `<a href="/doc.pdf">Read Paper</a>`, the model only translates `"Read Paper"`; the tag remains untouched in memory.
- **Recursive Bisection Resilience:** If Ollama returns malformed JSON or a count mismatch for a batch of N items, the function divides the array into halves and retries independently until isolating the problematic string.
- **In-Memory LRU Cache:** Retains fully translated JSON payloads for recent language requests in memory, avoiding redundant model inferences.

5.2 REST Endpoints

Method	Endpoint	Description	Status Code
`GET`	`/default-data`	Returns the canonical English master data tree (`INITIAL_PAGE_DATA`).	200 OK
`POST`	`/generate`	Translates or serves from cache the page data for a requested language.	200 OK (502 if Ollama fails)
`GET`	`/health`	Reports FastAPI operational health and active Ollama host/model status.	200 OK
```

6. Edge Infrastructure: Cloudflare Worker (`worker.js`)

The `worker.js` script acts as a high-speed reverse proxy and static asset server:

- **Static Hosting:** Delivers pre-built SPA bundles from `frontend/dist` as defined in `wrangler.jsonc`.
- **Unity Asset Streaming Proxy (CORS Bypass):**
  - `/unity/data`: Streams `VirtualTourVRweb.data` from GitHub Releases with `Access-Control-Allow-Origin: *` and `Content-Type: application/octet-stream` headers.
  - `/unity/wasm`: Streams `VirtualTourVRweb.wasm` with `Content-Type: application/wasm`.
- **360° Video Streaming with HTTP Range Support:** Routes `/unity/video360`, `/unity/crimson`, and `/unity/cereal` proxy high-definition MP4 videos, intercepting `OPTIONS` preflights and propagating `Range`, `Content-Range`, and `Content-Length` headers for zero-latency seeking in WebXR headsets.

7. Data Pipelines & Utilities (`tools/`)

- **`audit_publications.py`:** Bibliometric ETL script that ingests `papers.csv`, queries **Crossref** and **OpenAlex** APIs, downloads Open Access PDFs to `frontend/public/publication/` using SHA-1 hash names, and auto-generates `backend/base_data/sections/publication.py`.
- **`sync_people_photos.py`:** Scrapes high-resolution staff portraits from the official university site and outputs `people_photos.csv`.
- **`restore_publications.py`:** Standalone offline utility to rebuild publication structures from `manifest.csv` without internet connectivity.

8. Execution & Deployment Guide

8.1 Automated Local Startup

```
# Launch backend and frontend simultaneously with the helper script

./start_demo.sh
```

8.2 Manual Multi-Terminal Startup

```
# Terminal 1 — FastAPI Backend

cd backend

source venv/bin/activate

export OLLAMA_HOST="http://localhost:11434"
```

```
export OLLAMA_MODEL="translategemma:4b"

uvicorn main:app --host 0.0.0.0 --port 8000 --reload



# Terminal 2 — React Frontend

cd frontend

npm install

npm run dev
```

8.3 Cloudflare Workers Edge Deployment

```
cd frontend

npm run build

cd ..

npx wrangler deploy
```